

An abstract graphic on the left side of the slide. It features a close-up, angled view of a computer keyboard. Several keys are highlighted with glowing arrows pointing to the right. The arrows are in various colors: white, light blue, and orange. The background of the keyboard is dark, and the overall lighting is warm, with a gradient from dark blue at the top to orange at the bottom.

Base de datos nativas XML y MongoDB

Módulo: Acceso a datos

Ciclo: Desarrollo de Aplicaciones
Multiplataforma

- ♦ Del 17 de febrero al 7 ☐ Realización de ejercicios resueltos
 - ♦ Del 10 de febrero al 30 de marzo ☐ Realización del proyecto
 - ♦ Entrega del proyecto ☐ Día 30 de marzo
 - ♦ Día 1 de abril ☐ Examen UD 4
-

- ◆ Tipos de datos:
 - ◆ **Estructurados:** la información se ajusta a un formato □ Datos en una tabla
 - ◆ **Desestructurados:** no siguen un formato claro □ Documento de texto o vídeo
 - ◆ **Semi-estructurados:** tienen cierto formato que puede ir variando □ Página web
 - ◆ BDR son apropiadas para almacenar datos estructurados, sin embargo, son malas para almacenar datos no estructurados (□ BD XML).
 - ◆ XML:
 - ◆ Define el conjunto de reglas para organizar información de distintas maneras.
 - ◆ Características:
 - ◆ Lenguaje bien formado □ Todas las etiquetas cerradas
 - ◆ Fácil de leer
 - ◆ Extensible □ Puede ser ampliado con nuevas etiquetas
 - ◆ Autodescriptivo
-

- ◆ Tipos de documentos:
 - ◆ **Centrados en los datos:**
 - ◆ Muchos elementos pequeños con estructuras regular y bien definida
 - ◆ Ex.: Facturas, Pedidos, información de un alumno, etc.
 - ◆ **Centrados en texto:**
 - ◆ Pocos elementos con gran cantidad de texto con estructura de tamaño y contenido impredecible
 - ◆ Ex.: Libros, informes, memorias

- ◆ Opciones de almacenamiento de ficheros XML:
 - ◆ **Directo del fichero:** solo permite operaciones que proporciona el sistema de archivos.
 - ◆ **En una BD existente** (BDR, BDOO, BDOR):
 - ◆ **En una columna tipo binario grande** (BLOB): almacenamos el documento intacto
 - ◆ **Mapeo basado en tablas o en objetos:** requiere una transformación para ajustar la estructura a la BD
 - ◆ **BD Nativa XML:** almacenamiento y recuperación de forma intacta
 - ◆ SGBD BD Nativas:
 - ◆ **BD XML-compatibles:**
 - ◆ desglosan un documento en su modelo relacional
 - ◆ Tienen ya un modelo de datos □ Solo manejan documentos que encajen en su modelo
 - ◆ **BD XML Nativas:**
 - ◆ Respetan la estructura del documento
 - ◆ Proporcionan un modelo de datos propio □ Manejan todo tipo de documentos
-

BD Nativas XML: Características

- ◆ **Colección:** conjunto de documentos agrupados
 - ◆ Equivalencias: BD (tablas), Sistema de ficheros (directorio)
 - ◆ **Documentos:** contiene la información en formato XML
 - ◆ Equivalencias: BD (filas), Sistema de ficheros (fichero)
 - ◆ **Identificador único**
 - ◆ **Consultas:** Xquery
 - ◆ **Soporta transacciones**
 - ◆ **SG Nativos XML:**
 - ◆ **Comerciales:** TEXTML o TaminoXML Server
 - ◆ **Código abierto:** BaseX, eXistDB
-

```
<matriculas>
  <alumno>
    <nombre>Pedro</nombre>
    <apellidos>
      <apellido>López</apellido>
      <apellido>Ortega</apellido>
    </apellidos>
    <dni
pais="es">11112222A</dni>
  </alumno>
  <alumno>
    <nombre>Ana</nombre>
    <apellidos>
      <apellido>García</apellido>
    </apellidos>
    <apellido>Martínez</apellido>
  </alumno>
</matriculas>
```

- ♦ Obtener los nodos de los alumnos:
/matriculas/alumno
- ♦ Obtener los países:
/matriculas/alumno/dni/@país
- ♦ Obtener el nombre de los alumnos italianos
/matriculas/alumno[dni/@pais="it"]/nombre/text()
- ♦ Contar el número de alumnos
count(/matriculas/alumno)
- ♦ Obtener los apellidos de los alumnos con un nombre más largo que 3 caracteres
/matriculas/alumno[string-length(nombre/text())>3]/apellidos

XPATH: Funciones

Funciones XPATH				
abs()	avg()	boolean()	ceiling()	concat()
concat()	contains()	count()	distinct-values()	empty()
ends-with()	exists()	floor()	lower-case()	max()
min()	not()	replace()	remove()	string-length()
starts-with()	sum()	doc()		

- ◆ Cláusula FLWOR (For – Let – Where – Order by – Return)
- ◆ Los caracteres { y } delimitan expresiones que son evaluadas
- ◆ Mismas funciones que XPath

```
<ul>
  {
    for $x in doc("libros.xml")/biblioteca/libros/libro
    where $x/editorial = "O'Reilly"
    order by $x/titulo
    return <li>{data($x/titulo)}</li>
  }
</ul>
```

- ◆ Instalación BaseX
- ◆ Instalación y configuración del servidor

- ◆ Añadir un documento

- ◆ `session.add(String nombreBD, InputStream infoXML)`

- ◆ Reemplazar un documento

- ◆ `session.put(String nombreXML, InputStream infoNuevoXML)`

- ◆ Añadir un nodo (Xquery)

- ```
let $sup := <elemento2>Este es el segundo elemento</elemento2> return
insert node $sup as last into db:get("BDelementos", "elementos.xml")/raiz
```

- ◆ Eliminar un nodo (Xquery)

- ```
delete node nombreBD where condición
```

- ◆ Reemplazar un nodo (Xquery)

- ```
replace value of node ... with ...
```

# BaseX: Consultas

```
try(BaseXClient session = new BaseXClient("localhost", 1984, "admin", "admin")) {
 // Creación de la instancia de la consulta
 final String input = "for $i in 1 to 10 return <xml>Text { $i }</xml>";
 // Ejecución de la consulta
 try(Query query = session.query(input)) {
 // Comprobación e iteración de los resultados
 while(query.more()) {
 // Obtención del parámetro
 System.out.println(query.next());
 }
 // Imprimir la información de la consulta
 System.out.println(query.info());
 }
}
```